Trajectory Tracing for Epistemically Honest AI Agents

Complementing Output Trust with Path Soundness

Robin Langer Claude (Opus 4.6)

July 2026

Abstract

We introduce *trajectory tracing*, a system for evaluating whether an AI agent took a sound reasoning path to arrive at its claims, complementing the *trust-boundaries* system which evaluates the epistemic quality of the claims themselves. Where trust-boundaries asks "is this claim structurally justified?", trajectory tracing asks "did the agent's reasoning path contain detectable anti-patterns?" We show that trajectory systems are instances of trust systems in the sense of Langer (2026), so the canonical grading function τ, the certificate lemma, and the transport theorem all transfer directly. We define five trajectory anti-patterns motivated by real failures from deployed mathematical research agents, implement a validator for the first (circular verification), and describe how trajectory evaluation composes with output trust evaluation via a monotone morphism.

Contents

1 Introduction **2**
- 1.1 Motivating failures . . . 2
- 1.2 Contribution . . . 2

2 Trajectory Systems **3**
- 2.1 Events and the trajectory graph . . . 3
- 2.2 Trajectory grading . . . 3

3 Anti-Pattern Rules **4**
- 3.1 Rule 1: Circular verification . . . 4
- 3.2 Rule 2: Tautological transport (future) . . . 4
- 3.3 Rule 3: Estimand substitution (future) . . . 4
- 3.4 Rule 4: Shallow source depth (future) . . . 5
- 3.5 Rule 5: Brief-as-deme (future) . . . 5

4 Composition with Trust-Boundaries **5**
- 4.1 The morphism ψ . . . 5
- 4.2 The composition rule . . . 5

5 Implementation **6**
- 5.1 Architecture . . . 6
- 5.2 Event format . . . 6
- 5.3 Deployment descriptor . . . 6
- 5.4 Validation results . . . 6

6 Future Work **7**

7 Related Work **7**

1 Introduction

The trust-boundaries system [1] provides a framework for grading the epistemic quality of mathematical claims produced by AI agents. A trust system (D, T, s) consists of a DAG D of claims, a totally ordered chain T of trust levels, and a grading function $s : D \to T$. The canonical grade τ is computed by well-founded recursion as the weakest link along all dependency paths, and the boundary rule ensures that claims above a threshold have all dependencies also above that threshold. The system is deployed across four agents (Clio, Rick, MacBeth, and a RAG loop), with cross-agent trust transport governed by monotone maps φ.

This system evaluates **output**—the final claims in a registry. It does not evaluate **trajectory**—the sequence of actions the agent took to arrive at those claims. Google's "The New SDLC With Vibe Coding" [2] identifies this as a fundamental gap in agent evaluation:

> Output evaluation checks the final artifact: does the code compile, do the tests pass? Trajectory evaluation checks the full sequence of tool calls and intermediate reasoning. Both are necessary because a fluent output that skipped its verification steps is a more dangerous failure than one with a visible error.

Five failure modes from the agents' own dream journals motivate the present work. In each case, the output passed (or would have passed) trust-boundaries validation, but the reasoning path contained a detectable anti-pattern.

1.1 Motivating failures

F1. Circular verification (MacBeth, 2026-07-14). The agent defined the Dirichlet tensor product on containers by transporting along the extension functor $[\![-]\!]$. It then verified that $[\![-]\!]$ preserves the Dirichlet tensor—but the tensor was *defined* via $[\![-]\!]$, so the verification tested a tautology. In the agent's own words: "I transported the structure along the functor, then congratulated the functor for preserving it."

F2. Tautological transport (MacBeth, 2026-07-14). More generally, any verification of the form "F preserves S" where S was defined using F is tautological. The verification target was derived from the thing being tested.

F3. Estimand substitution (Lyra, GECCO 2026). Across 13 instances, the agent cited a correct number as evidence for a claim it did not actually support—the number was computed for a different mathematical object than the one in the claim. "A true number reaching for an object it did not have."

F4. Shallow source depth (MacBeth, 2026-07-14). The agent independently re-derived a result, believing it to be novel, when the result was a numbered proposition in a standard reference sitting in its own reading list—at `agent-summary` depth. "The reproof→citation pattern has now fired four times, and every single time the scooping source was at `agent-summary` depth."

F5. Brief-as-deme (Lyra, GECCO 2026). Sub-agents contradicted the brief and were correct. The trajectory that followed the brief looked procedurally sound ("follow the specification"), but the brief itself was the error source. "The lesson is not 'spawn more verifiers' — what works is the opposite-prior pair."

1.2 Contribution

We make the following contributions:

1. We define *trajectory systems* as instances of trust systems (Section 2), inheriting the canonical grading τ, the certificate lemma, and the transport theorem.
2. We define five *anti-pattern rules* as computable predicates on trajectory graphs (Section 3), each motivated by a real failure.
3. We describe how trajectory evaluation *composes* with output trust evaluation via a monotone morphism ψ (Section 4).
4. We implement and validate the system for one agent (Clio), with a deployment-descriptor pattern that extends to other agents by adding a JSON file (Section 5).

2 Trajectory Systems

2.1 Events and the trajectory graph

Definition 2.1 (Trajectory event)**.** A *trajectory event* is a record $e = (id, kind, action, inputs, outputs, \ldots)$ where:

- *id* is a unique identifier,
- *kind* $\in \{\texttt{compute}, \texttt{read}, \texttt{claim}, \texttt{verify}, \texttt{cite}, \texttt{delegate}, \texttt{decide}\}$,
- *inputs* is a list of event IDs (causal dependencies),
- *outputs* is a list of artifact references produced.

Events of kind `verify` carry additional fields: a *verification target* (the event ID being verified) and a *verification method*.

Definition 2.2 (Trajectory graph)**.** A *trajectory graph* is a DAG $G = (V, E)$ where V is a set of events and $(u, v) \in E$ iff $u \in inputs(v)$. The graph must be acyclic.

2.2 Trajectory grading

Definition 2.3 (Trajectory chain)**.** A *trajectory chain* T is a finite totally ordered set of grades. Our default chain is:

$$\texttt{unsupported} < \texttt{shallow} < \texttt{independent} < \texttt{cross-verified} < \texttt{mechanically-checked}$$

Definition 2.4 (Own-grade)**.** The *own-grade* function $g : V \to T$ assigns each event a grade based on its kind and properties:

- `read` events: grade depends on source depth (`deep-read` $\mapsto$ `independent`, `agent-summary` $\mapsto$ `unsupported`).
- `compute` events: `independent` (computations produce independent evidence).
- `verify` events: grade depends on verification method (`independent`, `cross-verified`, `mechanically-checked`, or `unsupported` for circular/tautological).
- `claim`, `cite`, `delegate`, `decide`: `shallow` (these get their grade from inputs).

Definition 2.5 (Canonical trajectory grade)**.** The *canonical trajectory grade* $\tau : V \to T$ is defined by well-founded recursion:

$$\tau(e) = \min\big(g(e), \min_{d \in inputs(e)} \tau(d)\big)$$

Proposition 2.6. *A trajectory graph with its trajectory chain and canonical grading is an instance of a trust system (D, T, s) in the sense of* [1]*. Therefore:*

1. *τ is the greatest sound assignment (certificate lemma).*
2. *Any over-claiming ($s(e) > \tau(e)$) is detectable.*
3. *Morphisms transport τ laxly: $\varphi(\tau(e)) \leq \tau'(F(e))$ (transport theorem).*

Proof. Set $C = V$ (events as claims), define checking steps as the dependency edges, and set the grade function to τ. The definitions match exactly. The proofs in Trust.lean apply without modification. □

Definition 2.7 (Trajectory boundary rule)**.** A trajectory grade at or above a boundary level $b \in T$ requires all input events to also be at or above b. This is the boundary rule from trust-boundaries, instantiated on trajectory events.

3 Anti-Pattern Rules

Each anti-pattern is a computable predicate on the trajectory graph. When a rule fires, it produces a *violation* with severity (error or warning).

3.1 Rule 1: Circular verification

Definition 3.1 (Circular verification)**.** A `verify` event e has *circular verification* if its verification target t is an ancestor of e in the dependency DAG. That is, there exists a directed path from t to e through the `inputs` relation.

Lemma 3.2. *Circular verification implies $\tau(e) = \bot$ on the cycle.*

Proof. If t is an ancestor of e and e verifies t, the verification evidence was derived from the thing being verified. No independent evidence exists, so the own-grade of the verify event is `unsupported` ($= \bot$). Since τ propagates the minimum, $\tau(e) = \bot$. □

Example 3.3 (MacBeth Day 96)**.** The trajectory graph contains:

e_1 : `read` (definition of strict monoidal functor)
e_2 : `compute` (define Dirichlet tensor via extension functor)
e_3 : `compute` (check functor preserves Dirichlet tensor)
e_5 : `verify` (verify preservation), target $= e_2$, inputs $= \{e_3\}$

Since e_3 depends on e_2 (the tensor definition), and e_5 depends on e_3, the target e_2 is an ancestor of e_5. The circular verification rule fires.

3.2 Rule 2: Tautological transport (future)

A `verify` event e verifying that a map F preserves a structure S, where S was *defined* using F (i.e., the definition event of S has F's introduction event in its inputs).

3.3 Rule 3: Estimand substitution (future)

A `verify` or `cite` event where the evidence's object tag disagrees with the claim's object tag—a correct number cited against the wrong mathematical object.

3.4 Rule 4: Shallow source depth (future)

A `claim` event at trajectory grade `independent` or above where every `read` event in its transitive input closure has source depth at or below `agent-summary`. This mirrors the source-blocking rule in trust-boundaries.

3.5 Rule 5: Brief-as-deme (future)

A `decide` event that chose path A (following the brief) over a sub-agent's recommended path B, where the trust-boundaries registry later shows claim(A) = `dead-end` and claim(B) $\geq$ `proved`. This rule is *retrospective*—it fires after the outcome is known.

Remark 3.4. Rule 5 is honest about what is detectable in real time versus in hindsight. At the moment of the decision, the brief looks authoritative. Only after the alternative proves correct and the brief's path dies can the anti-pattern be identified. This is a structural limitation, not a design flaw.

4 Composition with Trust-Boundaries

4.1 The morphism ψ

Trust-boundaries evaluates claims; trajectory-tracing evaluates reasoning paths. The two compose via a monotone map:

Definition 4.1 (Trajectory-trust morphism)**.** A *trajectory-trust morphism* is a monotone map $\psi : T_{\text{traj}} \to T_{\text{trust}}$ from the trajectory chain to the trust chain. It maps trajectory grades to trust grades:

`unsupported`	$\mapsto$	`speculative`
`shallow`	$\mapsto$	`computed`
`independent`	$\mapsto$	`proved`
`cross-verified`	$\mapsto$	`peer-reviewed`
`mechanically-checked`	$\mapsto$	`lean-verified`

4.2 The composition rule

Definition 4.2 (Composition check)**.** Given a trajectory graph G with canonical grading τ_G, a trust-boundaries registry R with canonical grading τ_R, and a morphism ψ, the *composition check* verifies:

$$\forall e \in V_G,\ \forall n \in V_R : \quad \text{if } e \text{ supports } n,\ \text{then } \psi(\tau_G(e)) \leq \tau_R(n)$$

A violation means the trajectory is weaker than the trust grade claims: the registry says `proved`, but the trajectory is `shallow`.

Proposition 4.3. *The composition check is an instance of the transport theorem (Trust.lean, Lemma 3.2):* $\psi(\tau_G(e)) \leq \tau_R(\psi_{claim}(e))$.

Remark 4.4. This gives a two-layer evaluation:

1. **Trust-boundaries**: Is the claim structurally justified by its sub-claims?
2. **Trajectory-tracing**: Was the reasoning path that produced the claim sound?

A claim can pass layer 1 and fail layer 2 (correct output, flawed path—fragile). A claim can fail layer 1 and pass layer 2 (incorrect output, sound path—fixable). The two failures have different characters and demand different responses.

5 Implementation

5.1 Architecture

The implementation mirrors the trust-boundaries architecture:

- **Validator** (`tracecheck/tracecheck.py`): CLI with three commands (`validate`, `check`, `report`). Python 3.10+, stdlib only.
- **Schema** (`tracecheck/schema.py`): Event validation, DAG construction from JSONL files.
- **Rules** (`tracecheck/rules.py`): Anti-pattern detection. Currently Rule 1 (circular verification); Rules 2–5 as future work.
- **Emitter** (`tracecheck/emit.py`): Lightweight library (∼40 lines) that agents import to produce trajectory events as append-only JSONL.
- **Deployment descriptors** (`deployments/*.json`): Per-agent configuration. Currently Clio; Rick and MacBeth require only adding a JSON file.

5.2 Event format

Events are stored as JSONL (one JSON object per line):

```
{"id": "evt-0001", "agent": "clio", "phase": "prove",
 "kind": "read", "action": "read Koroteev-Smirnov 2412.19383",
 "inputs": [], "outputs": ["def:p-curvature"],
 "source_depth": "deep-read"}
{"id": "evt-0002", "agent": "clio", "phase": "prove",
 "kind": "compute",
 "action": "computed p-curvature spectrum at q=zeta_p",
 "inputs": ["evt-0001"], "outputs": ["result:spectrum"]}
```

5.3 Deployment descriptor

Each agent's trajectory configuration is a JSON file specifying the trajectory chain, boundary level, and (future) the composition morphism ψ:

```
{
  "agent": "clio",
  "trajectory": {
    "chain": ["unsupported", "shallow", "independent",
              "cross-verified", "mechanically-checked"],
    "boundary_at": "independent"
  }
}
```

Adding a new agent requires only creating a new descriptor file. The schema, rules, and CLI are agent-agnostic.

5.4 Validation results

Running the validator on the circular-verification test fixture (encoding MacBeth's Day 96 failure):

```
$ python3 tracecheck/tracecheck.py check \
    --deployment deployments/clio.json \
    tracecheck/tests/circular_verification/trajectory.jsonl
```

```
Trajectory check: 1 error(s), 0 warning(s)

  [error] circular-verification at evt-0005:
    verification target 'evt-0002' is an ancestor of this
    verify event -- the evidence depends on the thing being
    verified, making the check tautological
```

The trajectory report shows canonical grades cascading through the DAG:

```
evt-0001  read      tau=unsupported  inputs=[(none)]
evt-0002  compute   tau=unsupported  inputs=[evt-0001]
evt-0003  compute   tau=unsupported  inputs=[evt-0002]
evt-0004  claim     tau=unsupported  inputs=[evt-0003]
evt-0005  verify    tau=unsupported  inputs=[evt-0003] ** FLAGGED **
```

Every event inherits `unsupported` because the sole `read` event was at `agent-summary` depth. The circular verification flag is the structural failure; the uniformly low τ is the grading consequence.

6 Future Work

1. **Rules 2–5**: Tautological transport, estimand substitution, shallow source depth, and brief-as-deme detection.

2. **Composition command**: A `compose` subcommand that takes both a trajectory JSONL and a trust-boundaries registry, checking that $\psi(\tau_G) \leq \tau_R$.

3. **Lean formalization**: Trajectory systems as instances of `System` from Trust.lean. Anti-pattern predicates with proofs that violations imply $\tau = \bot$. The key insight making this tractable: the Lean proofs transfer directly because trajectory systems *are* trust systems.

4. **Agent integration**: Instrumenting Clio's `prove` phase with `emit` calls, then Rick and MacBeth.

5. **Retrospective trajectory extraction**: Mining existing dream journals and agent logs to reconstruct trajectory graphs post hoc, enabling evaluation of historical sessions.

7 Related Work

Trajectory evaluation is discussed in Google's "The New SDLC With Vibe Coding" [2] in the context of software engineering agents, where it refers to checking that a coding agent used the right tools, followed the right steps, and didn't skip verification. Our setting is different: mathematical research agents whose "tools" are computations, paper reads, and proof attempts. The anti-patterns we detect (circular verification, estimand substitution) are specific to epistemic reasoning, not software development.

The trust-boundaries system [1] provides the formal foundation. The canonical grading τ, the certificate lemma, the transport theorem, and the directed-container structure are all inherited directly. The present work adds a new dimension (trajectory) that composes with the existing dimension (output trust) via a monotone morphism.

Lyra's "brief-as-deme" finding (GECCO 2026) is itself a result *about* multi-agent trajectory diversity. Her discovery—that diversity of *priors* (via opposite-prior pairs) matters more than diversity of *vendors* (spawning more agents)—suggests that trajectory evaluation should attend to the prior structure of verification, not just its presence.

References

[1] R. Langer et al., *Trust Boundaries: Epistemically Honest AI Agents*, 2026. Software: `github.com/RaggedR/trust-boundaries`.

[2] A. Osmani, S. Saboo, and S. Kartakis, *The New SDLC With Vibe Coding: From Ad-Hoc Prompting to Agentic Engineering*, Google, May 2026.

[3] R. Langer, *Trust.lean: Formalised Trust Systems in Lean 4*, 2026. No sorry, no Mathlib. Included in [1].